Certainly! Let's delve into the various topics outlined in your session, focusing on static members of a class, inheritance, access specifiers, constructors, overloading, hiding, overriding, sealed methods, abstract classes, abstract methods, and sealed classes in C#. Each section will include detailed explanations and examples to enhance your understanding.

1. Static Members of a Class

Static members belong to the class itself rather than to any specific instance. They can be accessed without creating an instance of the class.

1.1 Static Fields

- **Definition**: Shared across all instances of the class.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Counter
2{
3    public static int Count = 0;
4
5    public Counter()
6    {
7        Count++;
8    }
9}
10
11class Program
12{
13    static void Main()
14    {
15        new Counter();
16        new Counter();
17        Console.WriteLine(Counter.Count); // Output: 2
18    }
19}
```

1.2 Static Methods

- **Definition**: Can be called without creating an instance of the class.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class MathUtils
2{
3    public static int Add(int a, int b) => a + b;
4}
5
6class Program
7{
8    static void Main()
9    {
10        Console.WriteLine(MathUtils.Add(5, 10)); // Output: 15
11    }
12}
```

1.3 Static Properties

- **Definition**: Similar to static fields but can include logic in the getter/setter.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Configuration
2{
3    public static string AppName { get; } = "My Application";
4}
5
6class Program
7{
8    static void Main()
9    {
10        Console.WriteLine(Configuration.AppName); // Output: My
Application
11    }
12}
```

1.4 Static Constructors

- **Definition**: Used to initialize static members and called automatically.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Logger
2{
3    static Logger() => Console.WriteLine("Logger initialized.");
4
5    public static void Log(string message) => Console.WriteLine(message);
6}
7
8class Program
9{
10    static void Main()
11    {
12        Logger.Log("Application started."); // Logger initialized.
Application started.
13    }
14}
```

2. Static Classes

- **Definition**: A class that cannot be instantiated and can only contain static members.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public static class MathHelper
2{
3    public static int Square(int number) => number * number;
4}
5
6class Program
7{
8    static void Main()
9    {
10        Console.WriteLine(MathHelper.Square(5)); // Output: 25
```

```
11    }
12}
```

3. Static Local Functions

- **Definition**: Local functions that cannot capture variables from their enclosing scope.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Calculator
2{
3    public void Calculate(int number)
4    {
5        static int Factorial(int n) => n < 2 ? 1 : n * Factorial(n - 1);
6        Console.WriteLine(Factorial(number));
7    }
8}
```

4. Inheritance

- **Definition**: Allows a class to inherit members from another class.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Animal
2{
3    public void Speak() => Console.WriteLine("Animal speaks");
4}
5
6public class Dog : Animal
7{
8    public void Bark() => Console.WriteLine("Dog barks");
9}
```

5. Access Specifiers

- **Definition**: Control the visibility of class members.
- **Types**:
    - **public**: Accessible from anywhere.
    - **private**: Accessible only within the same class.
    - **protected**: Accessible within the same class and derived classes.
    - **internal**: Accessible within the same assembly.
    - **protected internal**: Accessible within the same assembly or derived classes.
    - **private protected**: Accessible only within the same class or derived classes in the same assembly.

6. Constructors in a Hierarchy

- **Definition**: Constructors are special methods called when an instance of a class is created.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class BaseClass
```

```
2{
3    public BaseClass() => Console.WriteLine("Base class constructor");
4}
5
6public class DerivedClass : BaseClass
7{
8    public DerivedClass() : base() => Console.WriteLine("Derived class
constructor");
9}
```

7. Overloading in Derived Classes

- **Definition**: Allows derived classes to define methods with the same name but different parameters.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class BaseClass
2{
3    public void Show() => Console.WriteLine("Base Show");
4}
5
6public class DerivedClass : BaseClass
7{
8    public void Show(int number) => Console.WriteLine($"Derived Show:
{number}");
9}
```

8. Hiding, Using New

- **Definition**: Hiding occurs when a derived class defines a method with the same name as a method in the base class using the `new` keyword.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class BaseClass
2{
3    public void Display() => Console.WriteLine("Base Display");
4}
5
6public class DerivedClass : BaseClass
7{
8    public new void Display() => Console.WriteLine("Derived Display");
9}
```

9. Override

- **Definition**: Allows a derived class to provide a specific implementation of a method that is already defined in its base class.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class BaseClass
2{
3    public virtual void Display() => Console.WriteLine("Base Display");
4}
```

```
5
6public class DerivedClass : BaseClass
7{
8    public override void Display() => Console.WriteLine("Derived
Display");
9}
```

10. Sealed Methods

- **Definition**: A method that cannot be overridden in any derived class.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public class BaseClass
2{
3    public virtual void Display() => Console.WriteLine("Base Display");
4}
5
6public class DerivedClass : BaseClass
7{
8    public sealed override void Display() => Console.WriteLine("Derived
Display");
9}
```

11. Abstract Classes

- **Definition**: A class that cannot be instantiated and may contain abstract methods that must be implemented in derived classes.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public abstract class Animal
2{
3    public abstract void Speak();
4}
5
6public class Dog : Animal
7{
8    public override void Speak() => Console.WriteLine("Bark");
9}
```

12. Abstract Methods

- **Definition**: Methods declared in an abstract class that do not have an implementation.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public abstract class Shape
2{
3    public abstract double Area();
4}
5
6public class Circle : Shape
7{
8    private double radius;
9    public Circle(double r) => radius = r;
10    public override double Area() => Math.PI * radius * radius;
```

```
11}
```

13. Sealed Classes

- **Definition**: A class that cannot be inherited.
- **Example**:

```
csharp
VerifyOpen In EditorRunCopy code
1public sealed class FinalClass
2{
3    public void Display() => Console.WriteLine("Final Class");
4}
```

Conclusion

This comprehensive overview covers static members, inheritance, access specifiers, constructors, method overloading, hiding, overriding, sealed methods, abstract classes, abstract methods, and sealed classes in C#. Understanding these concepts is crucial for effective object-oriented programming in C#. If you have any further questions or need clarification on any topic, feel free to ask!